Performance Best Practices

Use optimization strategies to build fast storefronts. Follow performance best practices for web fonts, resource hints, bundle optimization, and third-party scripts.

> **Note**
>
> This document is the entry point for performance topics. For in-depth documentation, see the topics listed next.

Topic	Key Areas
Data Fetching	Server-load everything, data classification, loaders, actions, fetchers, SCAPI request shape
Loading States	Suspense boundary granularity, skeleton vs. spinner, visual feedback patterns
State Management	URL state, context selector pattern, optimistic UI, avoiding derived state
Images	DIS integration, `<DynamicImage>`, `DynamicImageProvider`, responsive sources, alt text
Monitor the Performance of Storefront Next	Use performance tools to monitor the performance of Storefront Next and create a performance benchmark.

Optimize Web Fonts

To achieve optimal performance during page load, use system fonts or minimize the size of web fonts and improve their discovery. Large web font files take longer to download and negatively affect First Contentful Paint (FCP). An incorrect `font-display` value can cause layout shifts

Contents

Self-host web fonts instead of loading them from third-party CDNs like Google Fonts. Self-hosting eliminates cross-origin DNS lookups and connection setup, avoids browser cache partitioning (browsers isolate third-party CDN caches per site), and is required for GDPR compliance because loading fonts from external CDNs transmits the visitor's IP address to that third party on every page load. A 2022 ruling by the Munich Regional Court established this as a GDPR violation applicable across the EU. The alternative of gating external font loading behind a consent manager preserves CDN delivery but degrades the experience for users who haven't consented and adds implementation complexity. For details, see Google Fonts and GDPR.

Font Discovery

Browsers use `@font-face` to find fonts. Help the browser discover fonts earlier by inlining the `@font-face` declaration in the `<head>` and adding a `<link rel="preload">` directive. Without preload, the browser doesn't request the font until it computes a style that references it, adding a waterfall delay.

Font Download

Use the WOFF2 format for its superior compression. Prefer variable fonts because a single file covers multiple weights, reducing the number of requests and preload hints. Subset fonts to include only necessary characters when the full Unicode range isn't needed.

Font Rendering

The `font-display` CSS property controls how text is shown while a font loads. Use `swap` to immediately show a system fallback font and swap in the web font once loaded, avoiding Flash of Invisible Text (FOIT). Use `optional` to eliminate the swap-induced layout shift entirely. The web font is used only if it arrives before first render, otherwise the system font persists.

System Fonts

Using system fonts avoids the font download entirely and eliminates render-blocking. For examples of system fonts, see Fonts for Apple platforms and Windows 11 font list.

For more detail, see Optimize web fonts on web.dev.

downloads before they're needed, reducing latency when those resources are eventually requested.

The template renders resource hints in the `<head>` based on configuration values in `config.server.ts`, so they can be tuned per environment without code changes (`src/root.tsx`).

- `appConfig.links.preconnect`: Origins the browser should open early connections to (DNS + TCP + TLS). Use for services that will definitely be contacted on every page, such as the image CDN. The template preconnects to the DIS host by default.
- `appConfig.links.prefetchDns`: Origins for DNS-only prefetching. Lighter than `preconnect`, appropriate for services that may or may not be contacted (for example, analytics, optional third-party APIs).
- `appConfig.links.prefetch`: Specific resources to fetch and cache in the background. Use sparingly, as prefetched resources consume bandwidth regardless of whether the user navigates to them.

```
# Override via environment variables
PUBLIC__app__links__preconnect='["http
PUBLIC__app__links__prefetchDns='["htt
```

Warning

Only `preconnect` to origins that are actually used on every page. Each preconnect opens a TCP and TLS connection eagerly, so unused preconnects waste the browser's connection budget and can delay more important requests. Performance audits, such as Lighthouse's "Avoid unnecessary preconnects", flag this. If an origin is only used on some pages (for example, a payment provider on checkout), prefer `dns-prefetch` instead. DNS lookups are cheaper and don't trigger warnings when unused.

Bundle Optimization

Vite handles tree-shaking, minification, and chunk splitting automatically. Follow these practices help keep bundles small.

modals, drawers, rich editors, or heavy off-screen content, use `React.lazy()` with deferred mounting, split them into separate chunks that are loaded on demand. See Lazy Loading for Overlays for the pattern. For large route-specific component groups, use `manualChunks` in `vite.config.ts` to control how Rollup groups modules. The template uses this to split checkout components and per-locale translation files into dedicated chunks that are only loaded when needed.

- Analyze and monitor bundle size. Run `pnpm bundlesize:analyze` to generate an interactive visualization of client and server bundles (opens `build/client-bundle-size.html` and `build/ssr-bundle-size.html`). Run `pnpm bundlesize:test` to verify against configured size limits–CI enforces these checks on every PR.
- Avoid large dependencies for small tasks. Before adding a library, check its bundle size (for example, via bundlephobia). A 50 KB utility library for a function you could write in 10 lines is not a good tradeoff.
- Compression is handled automatically. Managed Runtime applies Gzip/Brotli compression to responses at the edge–no application-level configuration is needed.

React Rendering

Unnecessary re-renders inflate Interaction to Next Paint (INP) and degrade responsiveness. Here are the most impactful optimizations.

- Split React Contexts by concern. One context per domain (theme, locale, and user). A single large context re-renders all consumers on every value change–even consumers that don't use the changed value. See State Management.
- Memoize expensive computations. Use `useMemo` for derivations that are genuinely expensive. Don't memoize everything because the overhead of memoization exceeds the cost of cheap computations.
- Stabilize callback references. When passing callbacks to memoized child components, wrap them in `useCallback` to prevent the child from re-rendering on every parent render.
- Use `React.memo` selectively. Wrap components that re-render often with

Lazy Loading for Overlays (Modals, Drawers, and Dialogs)

Overlay components that are hidden on initial render, such as modals, drawers, and dialogs, must use `React.lazy()` with deferred mounting. Mount the `<Suspense>` subtree only after the first user interaction, not on page load. This keeps the overlay's code out of the main chunk entirely until it's actually needed, reducing page load size and Total Blocking Time (TBT).

```
1  const MyModal = lazy(() => import("@/c
2
3  function MyComponent() {
4    const [loaded, setLoaded] = useState
5    const [open, setOpen] = useState(fal
6
7    return (
8      <>
9        <Button
10         onClick={() => {
11           setLoaded(true);
12           setOpen(true);
13         }}
14       >
15         Open
16       </Button>
17       {loaded && (
18         <Suspense fallback={null}>
19           <MyModal open={open} onOpenC
20         </Suspense>
21       )}
22     </>
23   );
24 }
```

- `loaded` flips once on first click–controls when the chunk is fetched and the component mounts.
- `open` toggles visibility–re-opening after first load is instant.

they're bundled into the main chunk, increasing page load size and TBT.

Discouraged: `<Suspense><LazyComponent /></Suspense>` without a guard–the chunk is separate but still fetched and parsed on mount, adding to TBT during page startup.

Deferred Rendering with useDeferredRender

Mounting a `<Suspense>` boundary at the top of the component tree forces React to retain the entire fallback subtree in memory and process it during initial render, even if the user hasn't scrolled near that content. For large grids or heavy off-screen sections, this unconditional top-level mounting inflates Total Blocking Time (TBT) and competes for the main thread with LCP candidates in the initial viewport.

The `useDeferredRender` hook solves this main-thread contention by delaying the mount of a `<Suspense>` boundary until the browser reports an idle frame via `requestIdleCallback`. During that idle window, the component renders a lightweight skeleton placeholder instead of a live `<Suspense>` tree.

Three-Phase Rendering:

Phase	When	What renders
Pre-Idle	Immediately after page paint	Critical content and static skeleton placeholders. No `<Suspense>` boundary mounted.
Post-Idle (Pending)	After idle callback fires	`<Suspense>` boundary mounts with skeleton fallback. Stream begins.
Resolved	After Promise settles	Full content replaces skeleton.

```
  return (
    <>
      {/* Critical, initial-viewport c
      <ItemGrid items={criticalItems} 

      {/* Phase 1: no Suspense boundar
      {!shouldRender ? (
        <SkeletonGrid count={placehold
      ) : (
        /* Phase 2 & 3: Suspense bound
        <Suspense fallback={<SkeletonG
          <Await resolve={nonCriticalP
        </Suspense>
      )}
    </>
  );
}
```

Client-Side Transform Anti-Patterns

A common source of unnecessary CPU work is data computation that runs on every render when it should happen once in the server loader. Identify and move these transforms before they compound across component trees.

Anti-patterns to avoid:

- Building lookup maps from arrays in the component body. Variation matrices, facet lookups, and category trees constructed via `reduce` or nested loops are the most common offender–the work scales with input size and repeats on every render, compounding across every tile in a grid.
- Nested passes over the same data. `items.filter(...).map(...).sort(...)` or `items.map(item => other.find(...))` are O(n²) when repeated per render over larger collections. Derive the shape you need once, and index once.
- Deep spreads and clones. `items.map(item => ({ ...item, children: item.children.map(c => ({ ...c })) }))` allocates a fresh object tree every render and breaks prop equality for every child downstream.
- Repeated string normalization. Slug construction, URL parsing, or locale formatting that doesn't depend on client-only state belongs in the loader.

```
// ❌ BAD: lookup map rebuilt, and nes
function Grid({ items, categories }) {
```

```
    .map((c) => ({ ...c, items: byCate
    .sort((a, b) => a.label.localeComp
  return (
    <>
      {rows.map((row) => (
        <Row key={row.id} row={row} />
      ))}
    </>
  );
}

// ✅ GOOD: derive once in the loader;
export async function loader({ params
  const [items, categories] = await Pr
  const rows = buildRows(items, catego
  return { rows };
}

function Grid() {
  const { rows } = useLoaderData();
  return (
    <>
      {rows.map((row) => (
        <Row key={row.id} row={row} />
      ))}
    </>
  );
}
```

Promotional Pricing on Product Listing Pages

When displaying promotional pricing on a product listing page (PLP) or search results page, you have two options: use `expand=promotions` on SCAPI search calls, or pre-compute promotional prices into a custom sortable attribute via a scheduled job. The right choice depends on whether you sort results by promotional price.

Key point: `expand=promotions` calculates promotional prices at response time, after search results have already been sorted and returned. It cannot be used to sort results by promotional price.

Displaying Promotional Pricing with expand=promotions

To show promotional pricing information (promotional price, callout message, promotion ID) on your PLP alongside search results, use `expand=promotions`. It's straightforward and requires no additional setup, but it only enriches results – it doesn't influence their order.

On high-traffic PLPs, using `expand=promotions` adds per-request

Sorting by Promotional Price with a Custom Attribute

To sort search results by promotional price on high-traffic PLPs, use a custom sortable attribute populated by a scheduled job. This approach pre-computes promotional prices ahead of time so the search engine can sort by them natively with no per-request cost.

Combining Both Approaches

Combine both approaches if you sort by promotional price on a PLP and also want to show promotion details such as callout messages in the results. Use the custom sortable attribute to drive sort order, and `expand=promotions` to enrich the returned results with display data.

Third-Party Scripts

Third-party scripts, such as analytics, tag managers, A/B testing, chat widgets, and consent banners, are a common source of performance degradation. Each script adds to Total Blocking Time (TBT) and can delay Interaction to Next Paint (INP).

Keep these tips in mind for best performance results.

- Audit regularly. Every external script must justify its performance cost. Remove scripts that aren't actively used.
- Never load synchronously. Always use `async` or `defer`. A synchronous `<script>` blocks HTML parsing entirely.
- Lazy-load interaction-driven widgets. Chat widgets, social buttons, and similar components should load only when the user scrolls near them or clicks a placeholder–not on page load. See Lazy Loading for Overlays for the deferred mounting pattern.
- Use a Consent Management Platform (CMP). Integrate with a tag manager to prevent marketing and analytics tags from loading before user consent. This method satisfies privacy regulations and improves performance for users who haven't consented.
- Measure impact. Use the Chrome DevTools Coverage tab to identify unused JavaScript and CSS from third-party scripts.